Milestone Review
and
Design Pattern Introduction

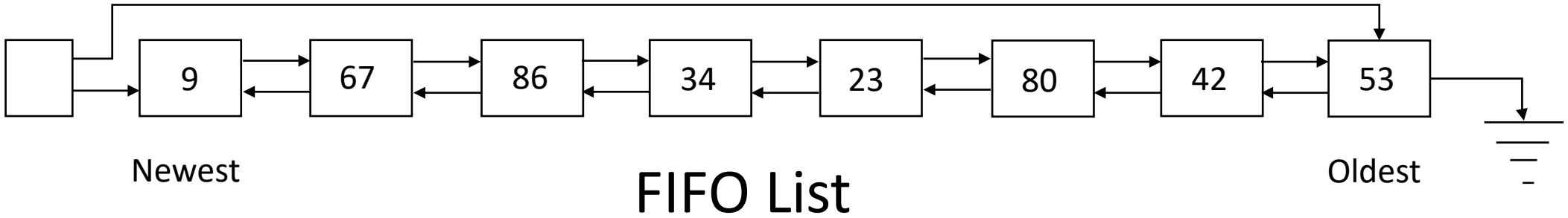
Milestone Redemption

- Students get 1 redemption per Milestone
- Resubmit any Milestone once to raise score
  - Full score is not possible (reduction in score)
- Compile
  - Submit all files as a unit on GitHub (even supplied files)
- Run unit tests

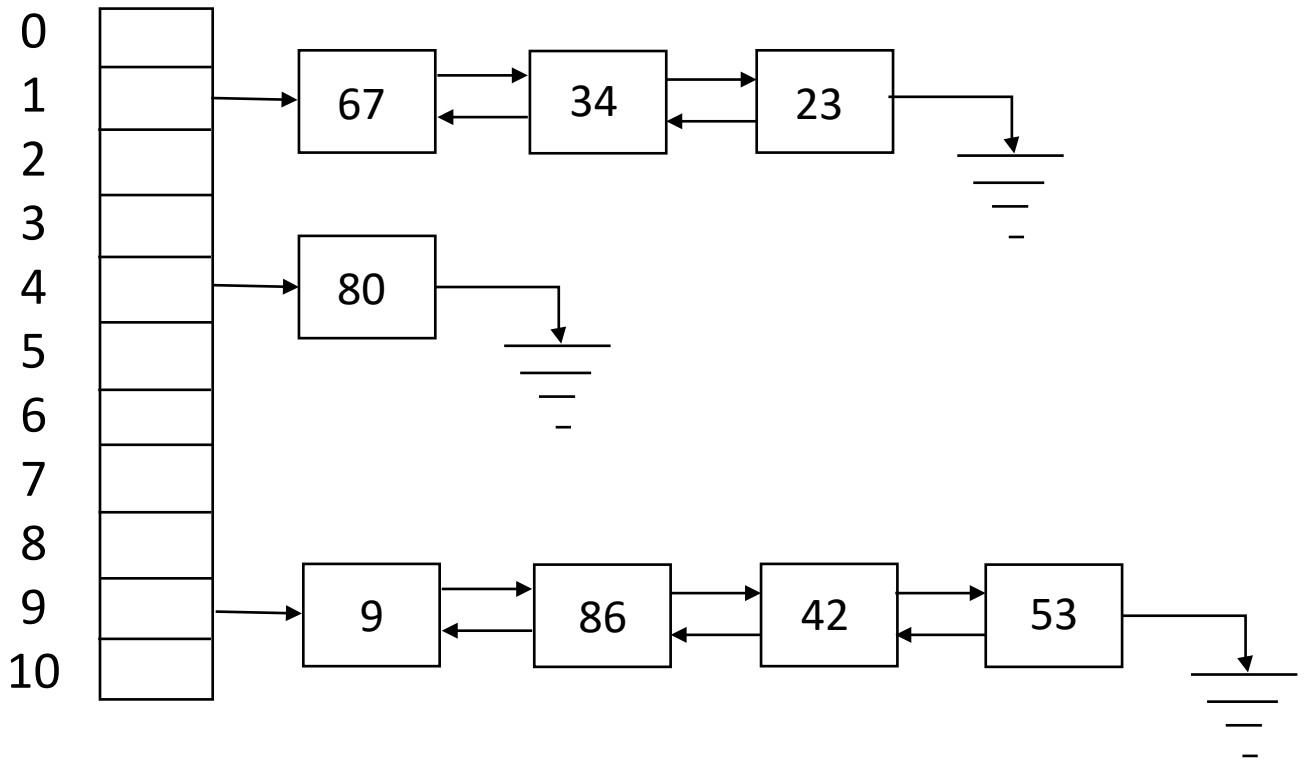
Milestone 1 – Doubly Linked List

- Very fast insert/delete
- Easy to move nodes
- Ordered list (NOT sorted by key)
- O(n) to find a key

- Used to implement First In First Out (FIFO) Queue



Milestone 2 - HashTable



- Very fast insert/delete
- Unordered list
- O(1) to find a key

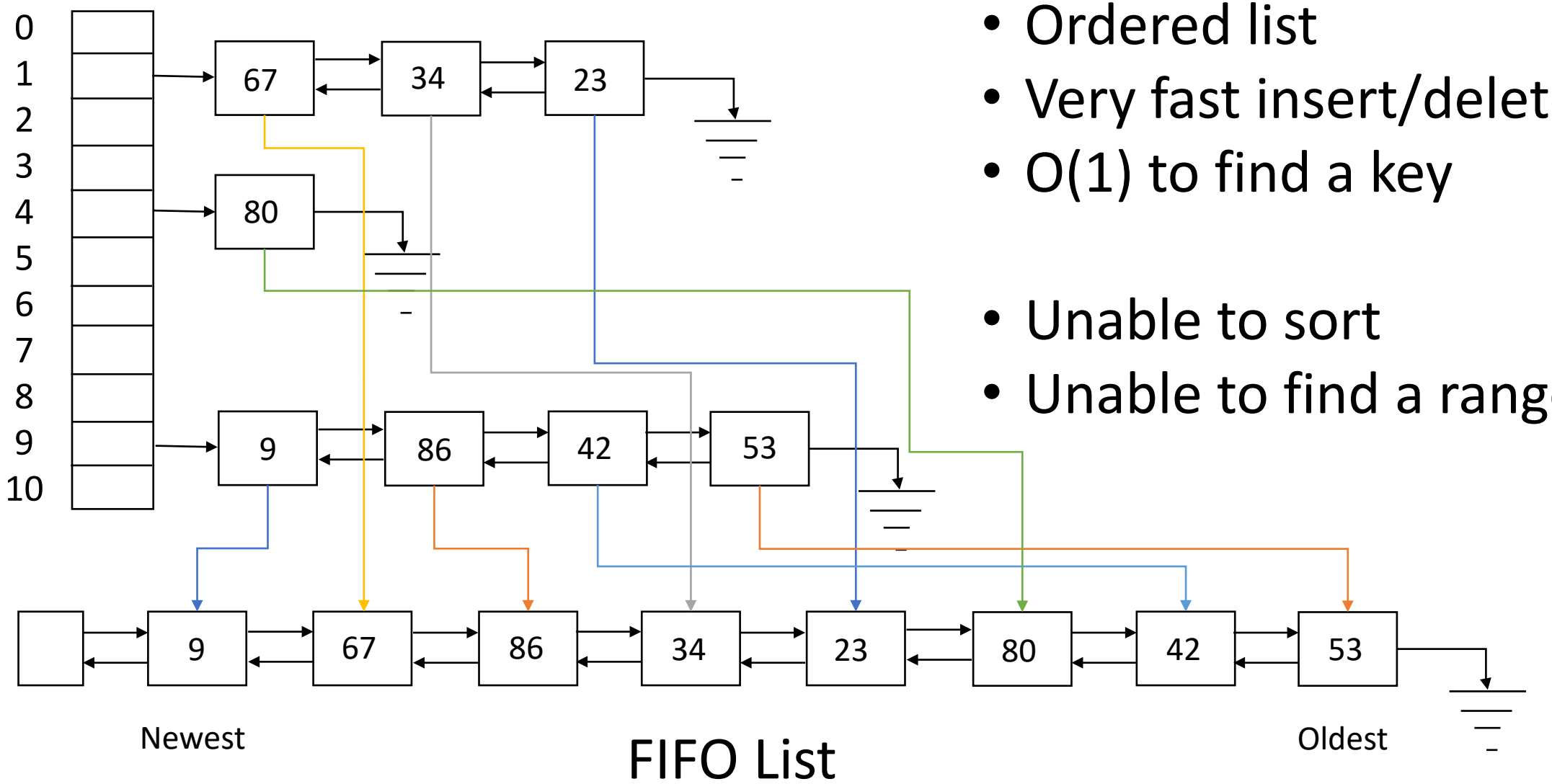
- Unable to sort
- Unable to find a range

FIFO and HashTable

- Both use Doubly Linked Lists (DLL)
- Each DLL has different data needs
  - FIFO nodes have a pointer to data (i.e., Employee, Student, etc.)
  - HashTable nodes will have a pointer to a FIFO node

- Several possible solutions
  1. Maintain 2 sets of DLL code (HashTable could be Singly Linked List)
  2. Node base class
  3. Template
  4. Standard Template Library

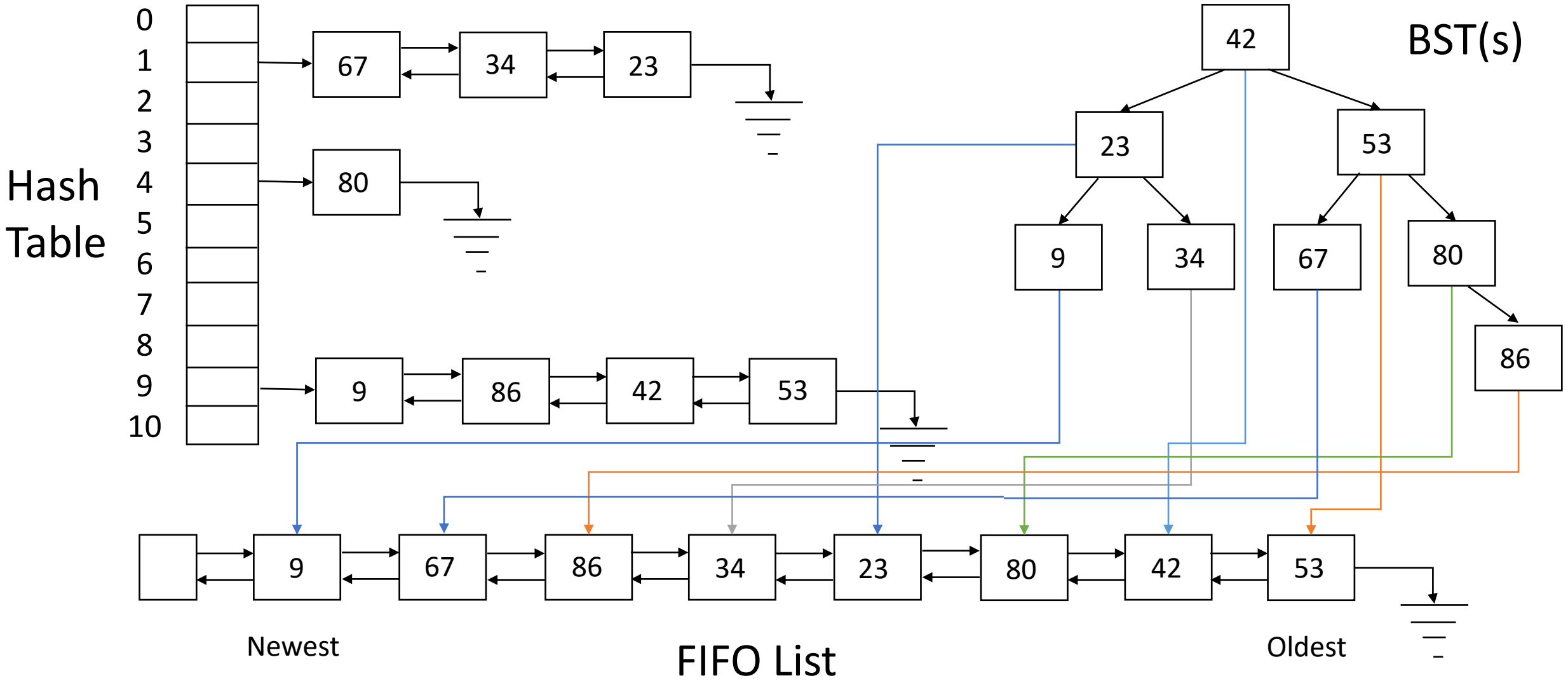
Milestone 3 – Cache Manager



- Ordered list
- Very fast insert/delete
- O(1) to find a key

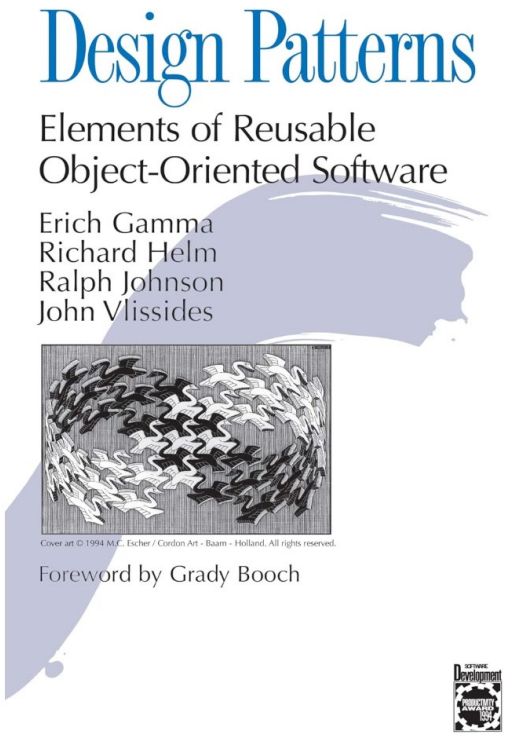
- Unable to sort
- Unable to find a range

Milestone 4 – Cache Manager (w/sort and range)



Introducing Design Patterns

- General, reusable solutions to common problems or challenges in software design
- Programming language independent
- Describes the structure, behavior, or interaction of software components for recurring design problems in a proven and efficient manner
- Multiple Data Structures working together to solve a problem
  - Important to understand when to use specific Data Structures (advantages/disadvantages)
- 23 classic design patterns, which are categorized into three broad types:
  - Creation - deal with object creation mechanisms, trying to create objects in a way that suits the situation
  - Structural - deal with how classes and objects are composed to form larger structures
  - Behavioral - focus on communication between objects and how they interact with each other; help manage responsibilities between objects and reduce dependencies
- Interview questions (Midterm 2 and Final)!



Design Patterns - Elements of Reusable Object-Oriented Software, 1994
Gamma, Helm, Johnson, Vlissides (Gang of Four)

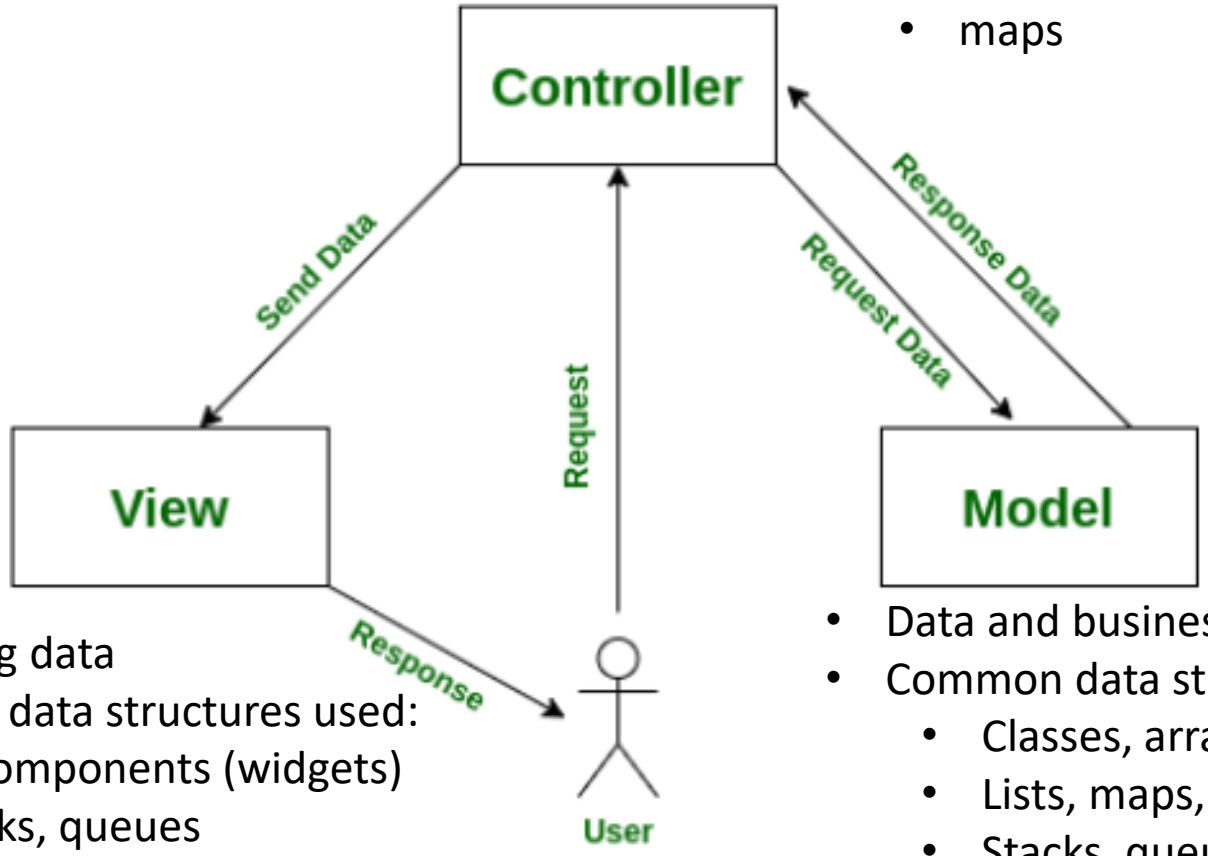
Data Structures and Design Patterns

- Data Structures:
  - Foundation of Software Development; All engineers who develop software must know Data Structures
  - Modern software cannot be developed without Data Structures
  - The grammar of software; all software are based on it for development
- Design patterns:
  - Provide proven solutions to common problems, allowing developers to reuse well-tested approaches
  - Shared vocabulary for developers, making it easier to discuss solutions to common problems
- All software products are based on Design Patterns
- All Design Patterns are based on Data Structures
- By using standard solutions, code becomes easier to understand and maintain

Behavioral - MVC

- Developed at PARC in late 70s
- Used by almost all modern UI's
- Separation of Concerns design principle
  - Distinct sections of functionality
- Some common uses:
  - Web application which updates a database
  - Mobile applications
- Highly used

- Intermediary for model and view
- Common data structures used:
  - Classes, queues, stacks
  - maps

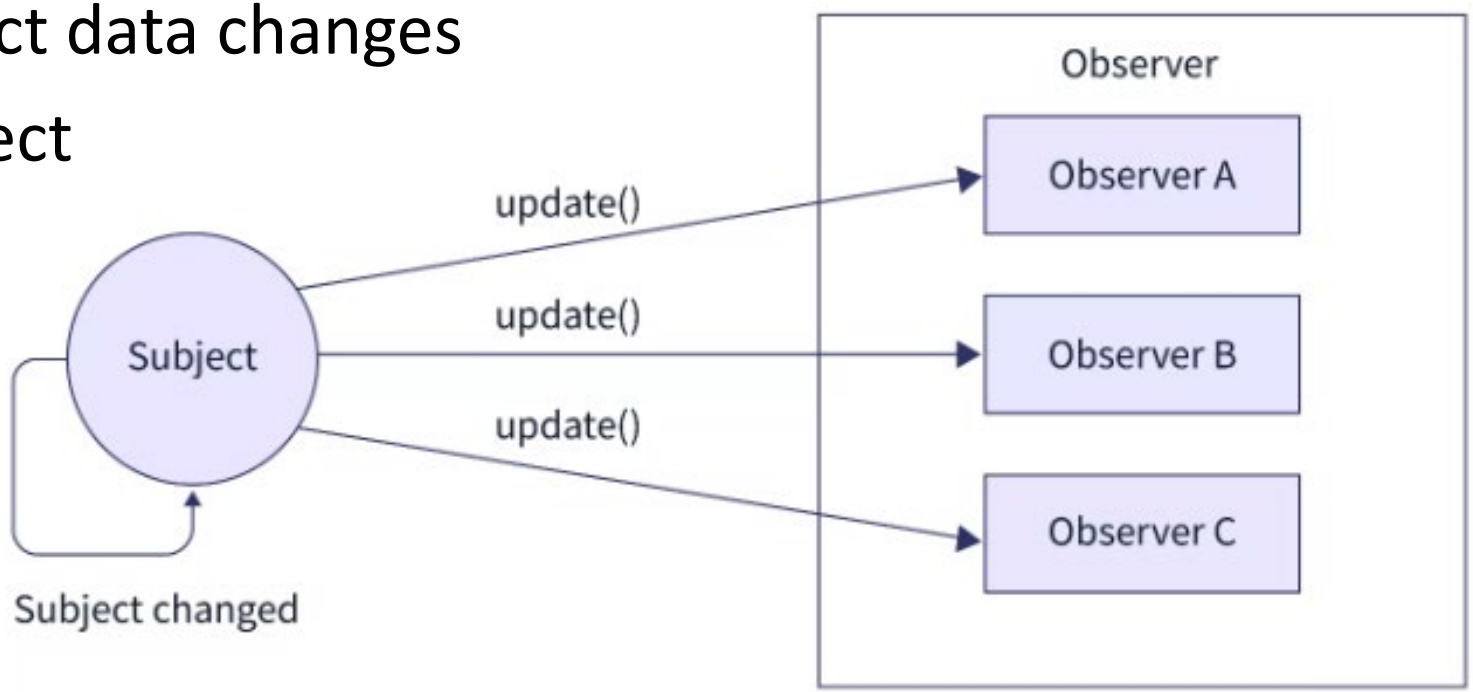


- Rendering data
- Common data structures used:
  - UI components (widgets)
  - Stacks, queues
  - Lists, trees

- Data and business logic
- Common data structures used:
  - Classes, arrays
  - Lists, maps, trees
  - Stacks, queues, graphs
- SQL tables

Behavioral - Observer

- Event pub/sub
- Used by modern UI's to reflect data changes
- Observers register with Subject
- Subject notifies Observers
  - Calls update()
- Some common uses:
  - News alerts
  - Stock alerts
- Highly used



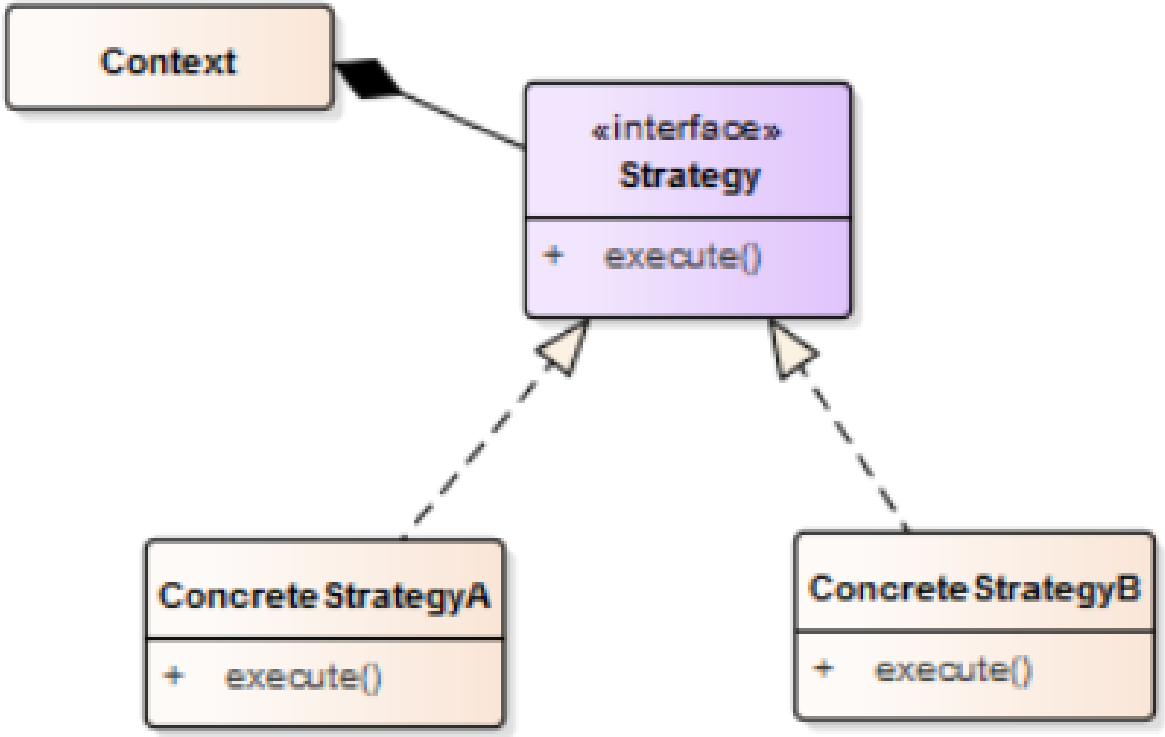
- Subject (observable) whose state changes
- Common data structures used:
  - Lists, arrays, queues
  - Set, map
  - Methods or functions (communications)

- Observer (listeners) subscribe to the subject; notified when subject state changes
- Common data structures used:
  - Classes
  - Methods or functions (event handler or callback when event occurs)

Behavioral - Strategy

- Define family of algorithms without changing the objects that use them
- Algorithm to execute is chosen at runtime
- Need different variants of an algorithm reflecting space/time trade-offs
- Some common uses:
  - Sort is dynamically selected based upon type or size of data
  - Payment systems (e.g., credit card, PayPal, bank transfer)
  - Discount calculation

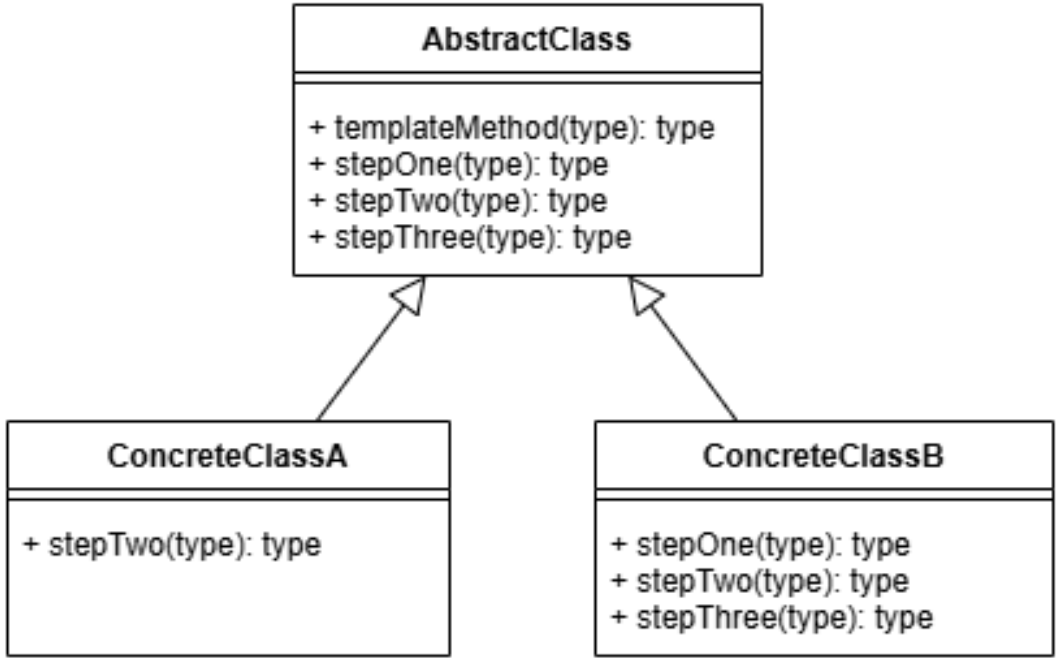
- Context (wrapper object) holds reference to strategy (can be dynamically swapped)
- Common data structures used:
  - Classes, map
  - List, array



Behavioral - Template

- Strategy - Polymorphism at its best
- Defines skeleton of an algorithm
- Some common uses:
  - Data processing (ETL – extract, transform and load) (e.g., read from database, process data, and write to CSV, JSON or XML)
  - Document processing (e.g., PDF, Word files, HTML)

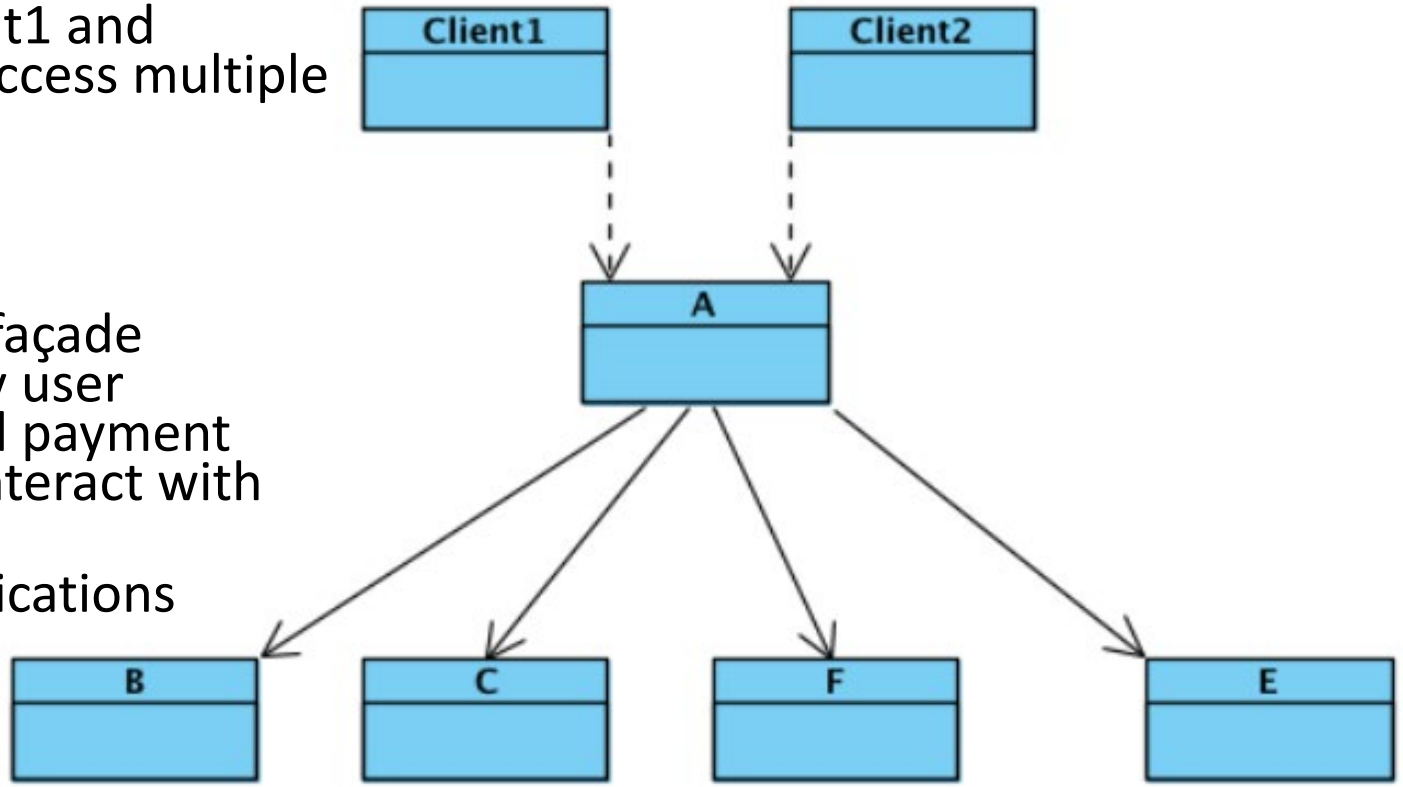
- Template (algorithm skeleton)
- Common data structures used:
  - Classes, map
  - List, array
  - Stacks, queues



Behavioral - Façade

- Provide simplified interface to complex subsystem
- Common data structures used:
  - Classes, map
  - List, array
  - Stacks, queues

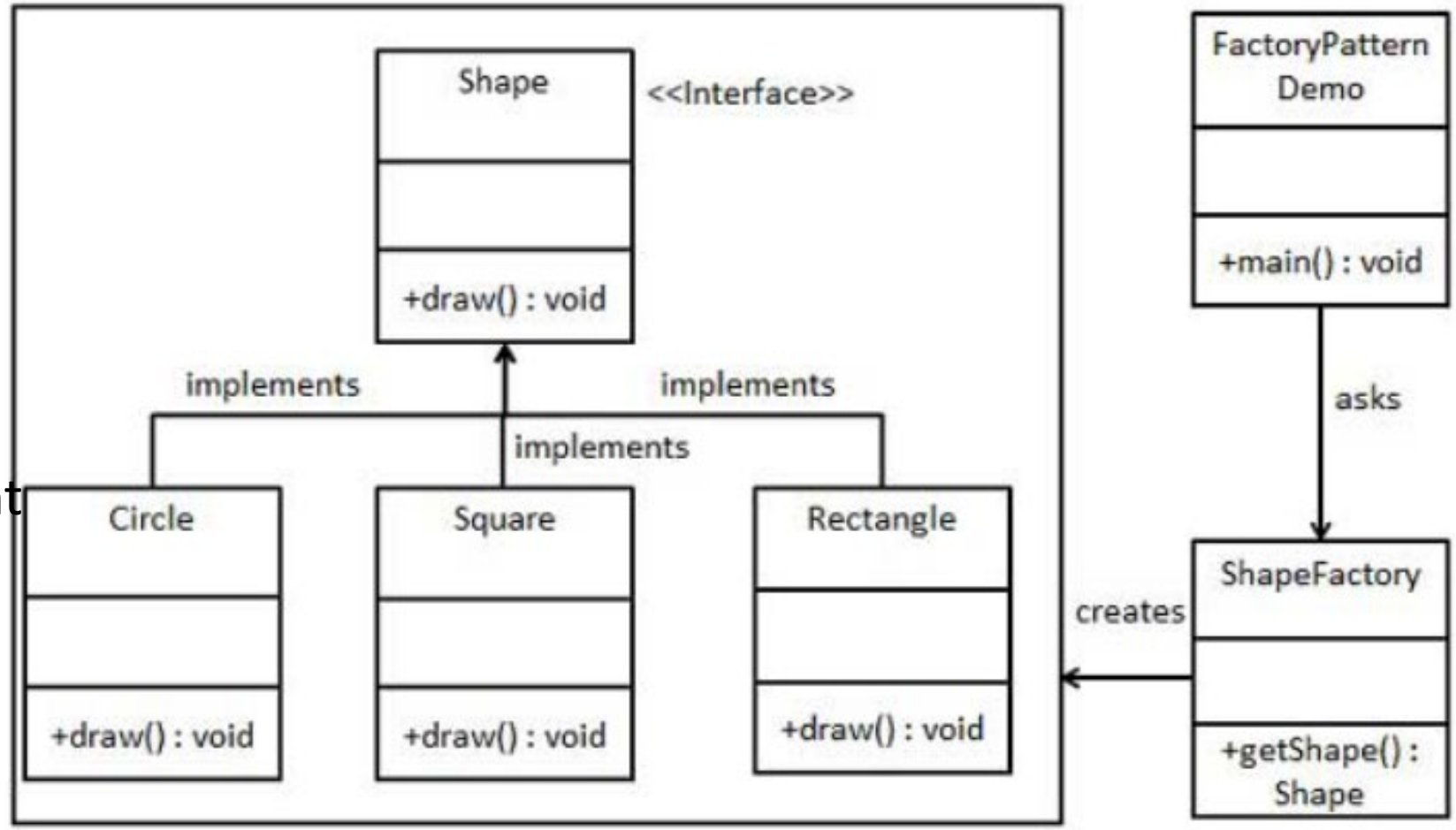
- JDBC, ODBC and DBI are good examples
- A is the JDBC, ODBC or DBI façade
- B, C, F, and E are different implementations
  - Allows different clients (i.e., Client1 and Client2) to be written once and access multiple databases
- Some common uses:
  - Database access
  - User interface design where the façade (wrapper) can be used to simplify user authentication, data retrieval and payment processing, instead if having UI interact with each service
  - Simplify subsystems in large applications



Creational - Factory

- Factory chooses which subclass to instantiate
- Allows for tracking of Object creation
- Some common uses:
  - Centralize logic for creating objects
  - Document management system that needs to create different document types (pdf, Word, etc.)
- Highly used

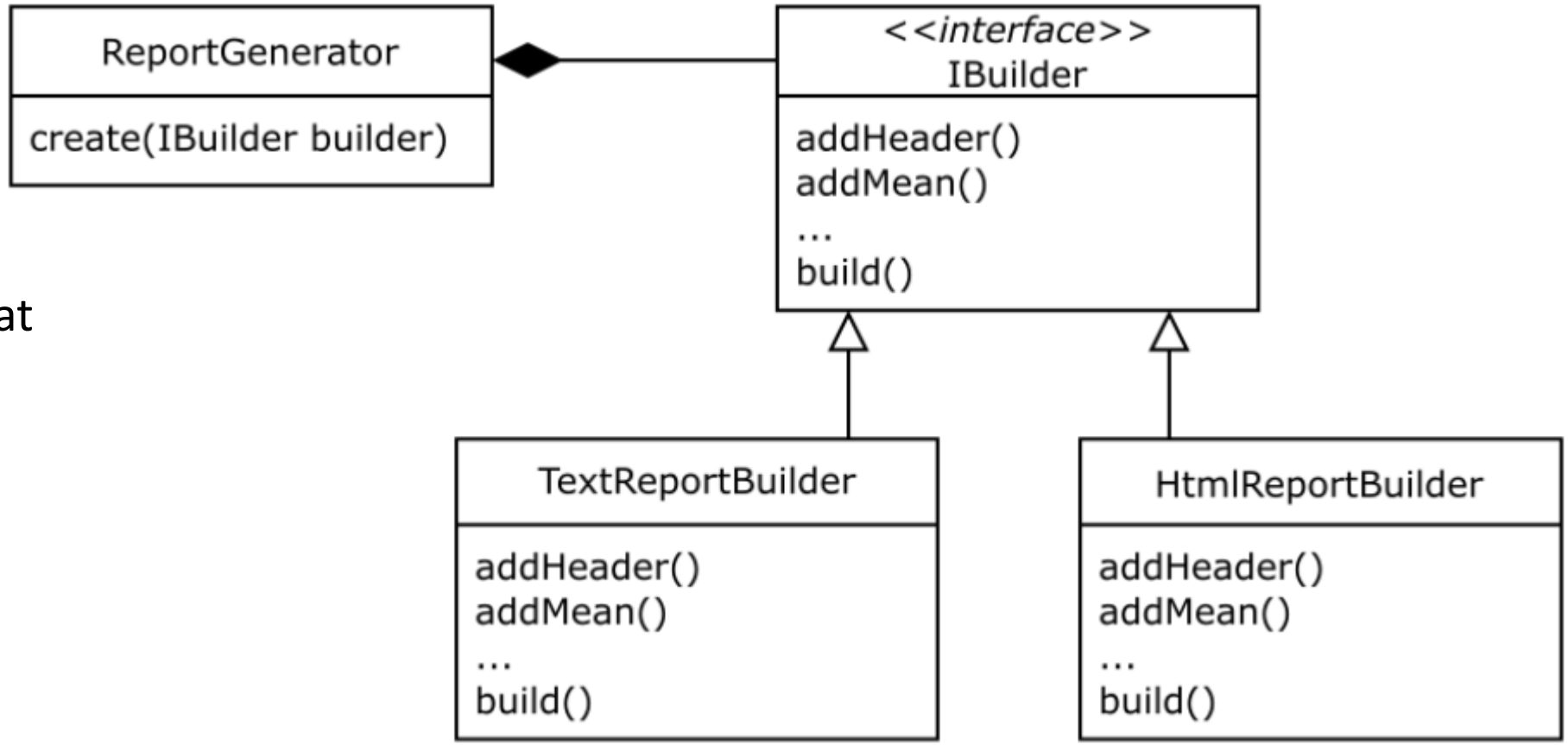
- Create objects without exposing the creation logic
- Common data structures used:
  - Classes, map
  - List, array
  - Interfaces/abstract classes



Creational - Builder

- Encapsulate complex object creation
- Delegate object creation to Builder
- Create object with many attributes or parameters
- Some common uses:
  - Create a document that has same content but different structural representations (e.g., XML, JSON)

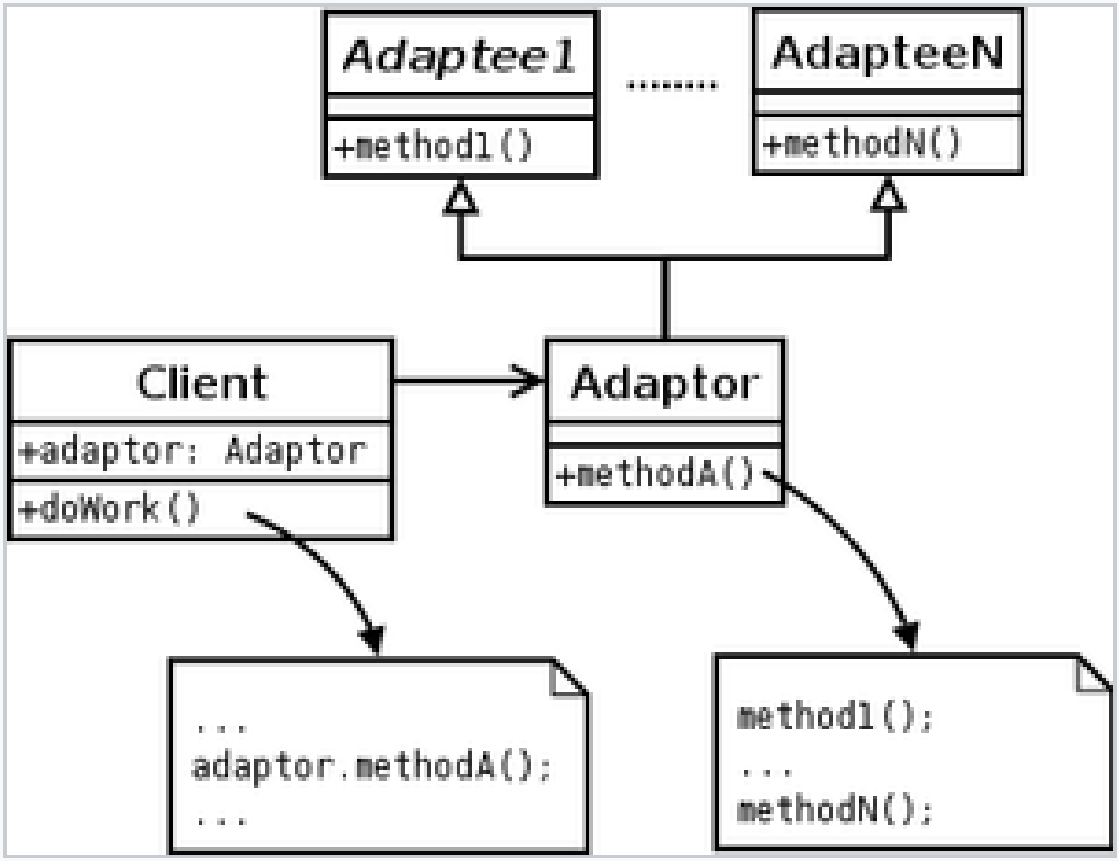
- Create different representations of an object using the same construction process
- Common data structures used:
  - Classes, map
  - List, array



Structural - Adapter

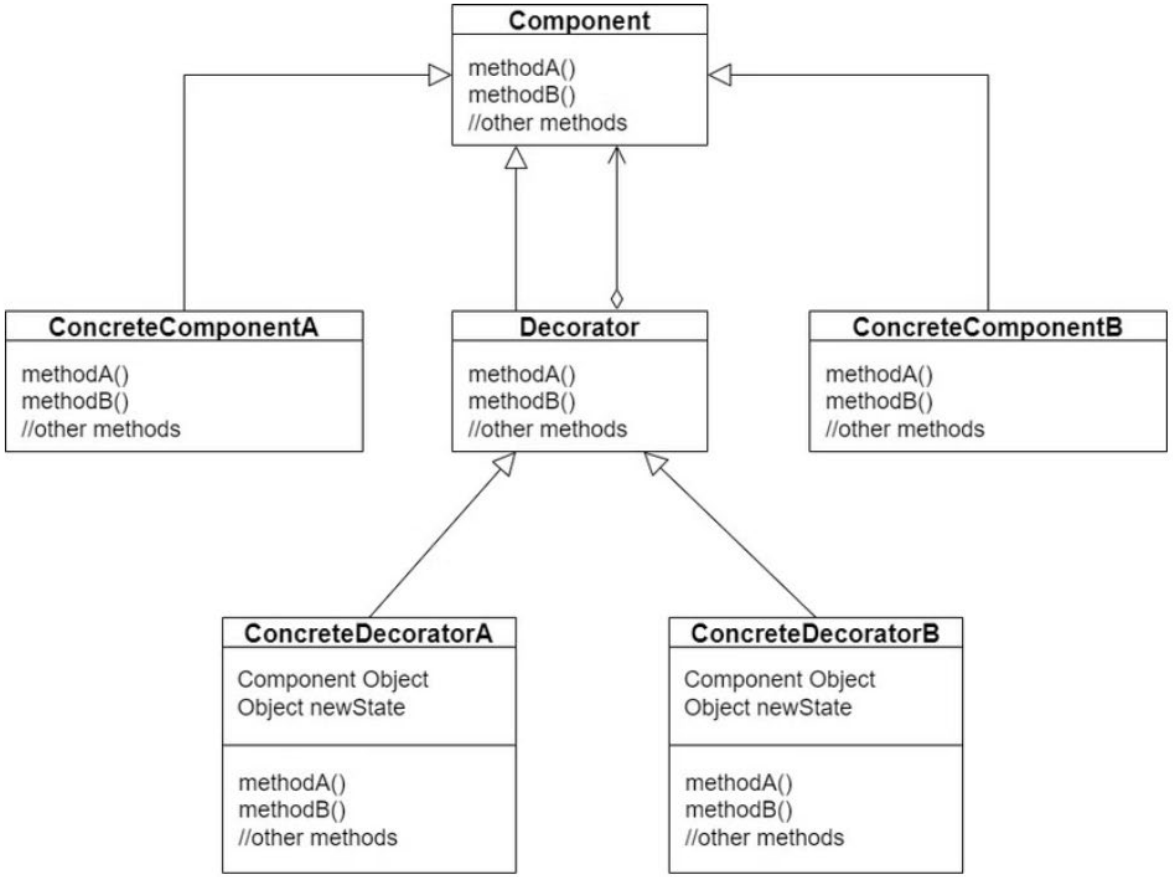
- Wrapper classes
- Converts interfaces
- Allows incompatible interfaces to work together
- Some common uses:
  - Integrate legacy systems with new code, e.g., old payment gateway to new user interface
  - Adapting a json API to an XML API
  - Converting data formats (e.g., XML to JSON or vice versa)

- Enable an interface of a class to be used as another interface; allows incompatible interfaces to work together
- Common data structures used:
  - Classes, map
  - List, array
  - Queues, stacks, buffers



Structural - Decorator

- Allows new functionality to be added to an object dynamically
- Open/Closed design principle
  - Open for extension
  - Close for modification

- Extension Options
  - Inheritance
    - Is a relationship
  - Composition
    - Has a relationship
    - Note that decorators have Component embedded
- Some common uses:
  - Add borders, shadows or other visual effects in a graphical application
  - Add logging, caching or transaction management, but don't want to modify original class implementation

- Create flexible and reusable where new functionality can be wrapped around existing behavior in object-oriented way
- Common data structures used:
  - Classes, map
  - List, sets
  - Queues, stacks